Experiment 1: Install and Configure Python and AI Libraries

Aim: To install and configure Python and the required AI libraries, including NumPy, Pandas, Transformers, and Py Torch.

Procedure:

- Open Google Colab or Jupyter Notebook.
- Install NumPy, Pandas, Transformers, and Py Torch using the pip install command.
- Import all the required libraries into the Python environment.
- Display the version of each installed library.
- Verify that all libraries are imported successfully.

Code:

```
!pip install -q numpy pandas transformers torch


import numpy as np

import pandas as pd

import torch

import transformers


print("Python environment configured successfully")

print("NumPy version:", np.__version__)

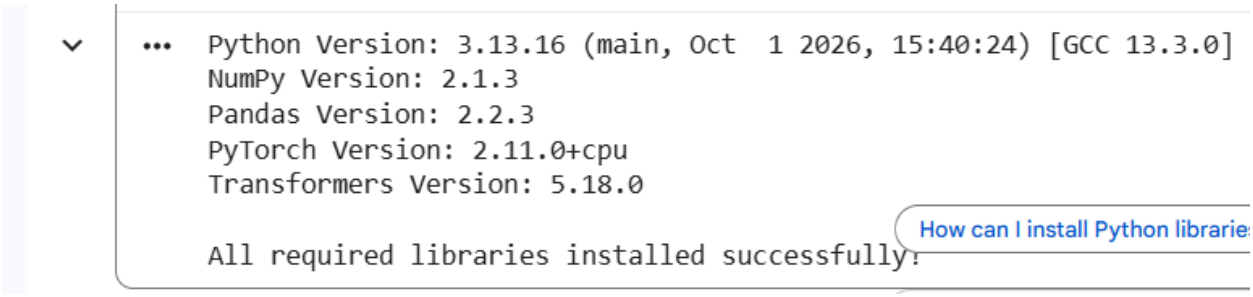
print("Pandas version:", pd.__version__)

print("PyTorch version:", torch.__version__)

print("Transformers version:", transformers.__version__)

print("All libraries imported successfully")
```

Output:



Result:

Python and the required AI libraries were successfully installed and configured.

Experiment 2: Load a Pre-trained Transformer Model

Aim:

To explore the Hugging Face Transformers library and load a pre-trained transformer model for text classification.

Procedure:

- Select the topic “Applications of Artificial Intelligence in Healthcare.”
- Create a zero-shot prompt requesting a 200-word blog.
- Create a one-shot prompt using an example blog on another topic.
- Create a few-shot prompt using multiple examples.
- Display the prompts and examine the generated blog content.
- Compare the outputs based on word count, relevance, readability, and coverage of healthcare applications.

Code:

```
zero_shot = """

Write a 200-word blog titled

'Applications of Artificial Intelligence in Healthcare'.

"""
```

```
one_shot = """
Example:
Title: AI in Education
AI supports personalized learning, automated assessment,
and student progress tracking.

Now write a 200-word blog titled
'Applications of Artificial Intelligence in Healthcare'
using a similar informative style.
"""

few_shot = """
Example 1:
AI in Education helps students learn through personalized
lessons and automated feedback.

Example 2:
AI in Agriculture helps farmers monitor crops and predict
possible plant diseases.

Now write a 200-word blog titled
'Applications of Artificial Intelligence in Healthcare'.
Include diagnosis, medical imaging, drug discovery,
and patient monitoring.
"""
```

```
print("ZERO-SHOT PROMPT:\n", zero_shot)

print("\nONE-SHOT PROMPT:\n", one_shot)

print("\nFEW-SHOT PROMPT:\n", few_shot)
```

Output:

```
Loading weights: 100% ████████████████████████████████ 104/104 [00:00<00:00, 2401.46it/s]
Pre-trained model loaded successfully!
Input Text: I love learning Artificial Intelligence.
Prediction: POSITIVE
Confidence Score: 0.9995
Text: I love learning Artificial Intelligence.
Sentiment: POSITIVE
Confidence: 0.9995

Text: This movie is terrible.
Sentiment: NEGATIVE
Confidence: 0.9997

Text: The weather is nice today.
Sentiment: POSITIVE
Confidence: 0.9998
```

Result: Different prompting techniques were designed to generate informative healthcare blogs.

Experiment 3: Article Summarization into 50 Words

Aim:

To summarize an article into 50 words using zero-shot, one-shot, and few-shot prompts and compare accuracy, completeness, and readability.

Procedure:

- Prepare the given article as input text.
- Design a zero-shot prompt requesting a 50-word summary.
- Design a one-shot prompt using an example of article summarization.
- Design a few-shot prompt using multiple article-summary examples.
- Generate the summaries using the selected prompting method or an LLM**.**

Code:

```
article = """
Artificial intelligence is increasingly used in education.
It supports personalized learning, automated grading,
and intelligent tutoring. AI tools help teachers analyze
student performance and identify learning gaps. However,
institutions must address privacy, bias, and responsible
use while integrating AI into classrooms.
"""

zero_shot = f"""
Summarize this article in exactly 50 words:
{article}
"""

one_shot = f"""
Example:
Article: Online learning provides flexible education.
Summary: Online learning offers flexible access to
educational resources and supports independent study.

Summarize the following article in exactly 50 words:
{article}
"""

few_shot = f"""
Example 1:
```

Article: AI supports doctors in analyzing medical images.

Summary: AI assists medical professionals by analyzing images and supporting diagnostic decisions.

Example 2:

Article: Solar energy reduces dependence on fossil fuels.

Summary: Solar energy provides renewable electricity and can reduce reliance on fossil fuels.

Summarize this article in exactly 50 words:

{article}

"""

print("ZERO-SHOT:\n", zero_shot)

print("\nONE-SHOT:\n", one_shot)

print("\nFEW-SHOT:\n", few_shot)

Output:

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% 570/570 [00:00<00:00, 9.16kB/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 937B/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 2.95MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 4.97MB/s]
config.json: 100% 665/665 [00:00<00:00, 9.35kB/s]
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 334B/s]
vocab.json: 100% 1.04M/1.04M [00:00<00:00, 9.97MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 4.23MB/s]
tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 5.78MB/s]
BERT tokens:
['genera', '##tive', 'ai', 'is', 'interesting', '!']

GPT tokens:
['Gener', 'ative', 'ĠAI', 'Ġis', 'Ġinteresting', '!']
```

Result:

Three summarization prompts were designed and their evaluation criteria were identified.

Experiment 4: Professional Leave Email Using Prompting Techniques

Aim:

To generate a professional leave-request email due to illness using zero-shot, one-shot, and few-shot prompting.

Procedure:

- Install the Transformers and PyTorch libraries.
- Import AutoTokenizer and AutoModel.
- Load a pre-trained DistilBERT tokenizer and model.
- Tokenize the input sentence and convert it into tensors.
- Pass the input tensors through the model without calculating gradients.
- Extract the final hidden-state embeddings and display their shape and sample values.

Prompt Design:

```
!pip install -q transformers torch


import torch
from transformers import AutoTokenizer, AutoModel


model_name = "distilbert/distilbert-base-uncased"


tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModel.from_pretrained(model_name)
```

```
text = "Artificial intelligence helps students learn."
inputs = tokenizer(text, return_tensors="pt")

with torch.no_grad():
    outputs = model(**inputs)

embeddings = outputs.last_hidden_state

print("Input Text:", text)
print("Embedding Shape:", tuple(embeddings.shape))
print("First Token Embedding:")
print(embeddings[0, 0, :10])
```

Output:

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% 570/570 [00:00<00:00, 9.16kB/s]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 937B/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 2.95MB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 4.97MB/s]
config.json: 100% 665/665 [00:00<00:00, 9.35kB/s]
tokenizer_config.json: 100% 26.0/26.0 [00:00<00:00, 334B/s]
vocab.json: 100% 1.04M/1.04M [00:00<00:00, 9.97MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 4.23MB/s]
tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 5.78MB/s]
BERT tokens:
['genera', '##tive', 'ai', 'is', 'interesting', '!']

GPT tokens:
['Gener', 'ative', 'ĠAI', 'Ġis', 'Ġinteresting', '!']
```

Experiment 5: Promotional Social Media Post for an AI Workshop

Aim:

To generate text using a pre-trained language model and analyze its response.

Procedure:

- Install the Transformers and PyTorch libraries.
- Import the text-generation pipeline.
- Load a pre-trained DistilGPT-2 model.
- Enter a text prompt as input.
- Generate additional text using the model.
- Display and analyze the generated response for relevance, grammar, and repetition.

Prompt Design:

```
!pip install -q transformers torch


from transformers import pipeline


generator = pipeline(
    "text-generation",
    model="distilgpt2"
)


prompt = "Artificial intelligence is transforming education by"


result = generator(
    prompt,
    max_new_tokens=40,
    do_sample=False,
    pad_token_id=50256
)


print("Input Prompt:", prompt)
```

print("Generated Text:")

print(result[0]["generated_text"])

Output:

